SOFTWARE PROJECT MANAGEMENT REPORT

Inventory Management System

A full-stack MERN project designed for inventory visibility, stock control, movement tracking, and low-stock awareness through a soft-tone responsive web interface.

Project Type

Full-stack MERN web application

Technology Stack

MongoDB, Express, React, Node.js, Vite, JWT

Prepared For

Inventory Management System academic project documentation

Date

08 April 2026

EXPERIMENT-01

Aim

To study, analyze, design, and document the requirements of an Inventory Management System by preparing a detailed problem statement, Software Requirements Specification (SRS), and UML diagrams for a MERN-based solution.

Theory

A Software Requirements Specification is the formal foundation of a software project. It defines what the system must do, who will use it, the environment in which it will operate, the quality standards it must maintain, and the assumptions under which development will proceed. A clear SRS reduces ambiguity, prevents scope drift, and gives developers, testers, and stakeholders a shared understanding of expected behavior.

For an inventory management system, the SRS is especially important because stock accuracy, movement history, supplier tracking, and replenishment awareness all depend on consistent workflows and reliable data handling.

Problem Statement

Traditional inventory processes often rely on spreadsheets or manual registers. This leads to duplicate entries, delayed updates, missed reorder cycles, and poor visibility into stock movement. Teams struggle to answer basic operational questions such as what is available, what is running low, what has moved today, and what the current inventory value is.

The proposed Inventory Management System addresses these issues by providing a centralized digital platform where authorized users can log in, manage products, organize items by category, record stock-in and stock-out transactions, monitor low-stock alerts, and review recent movement activity in real time.

> **Software Requirements Specification**

1. Introduction

1.1 Purpose: To describe the functional and non-functional requirements of the Inventory Management System.

1.2 Scope: The system is a web application for maintaining product records, stock counts, supplier details, locations, and movement logs.

1.3 Definitions: SRS - Software Requirements Specification, UI - User Interface, API - Application Programming Interface, JWT - JSON Web Token, CRUD - Create Read Update Delete.

1.4 Overview: This document covers system description, features, constraints, interfaces, planning artifacts, metrics, and cost estimation.

2. Overall Description

2.1 Product Perspective: The application follows a client-server architecture where the React frontend communicates with an Express API, which in turn persists data in MongoDB.

2.2 Product Functions: User authentication, category management, product creation, stock editing, movement logging, dashboard insights, and low-stock monitoring.

2.3 User Classes: Admin users manage products and operational settings; staff users update stock entries and observe inventory status.

2.4 Operating Environment: Web browsers on Windows, Linux, and macOS; Node.js server runtime; MongoDB database server.

2.5 Constraints: Stable network access is required, JWT secrets must be protected, and MongoDB must be reachable for persistent storage.

2.6 Assumptions: Users enter accurate stock data, supplier information is known, and the organization follows defined stock workflows.

3. Functional Requirements

- The system shall allow users to register and log in securely.
- The system shall allow authorized users to create and update product records.
- The system shall organize products by category, supplier, and storage location.
- The system shall record stock-in, stock-out, and adjustment movements.
- The system shall recalculate product stock status after each movement.
- The system shall display low-stock and out-of-stock conditions on the dashboard.
- The system shall show inventory value based on quantity and unit cost.
- The system shall provide recent movement history for traceability.

4. Non-Functional Requirements

- Performance: dashboard and list views should respond quickly for normal warehouse usage.
- Security: authentication must use password hashing and token-based authorization.
- Usability: the UI should be responsive, soft-toned, and easy to navigate.
- Maintainability: frontend and backend modules should be separated cleanly.
- Scalability: the data model should support expansion to more products and categories.

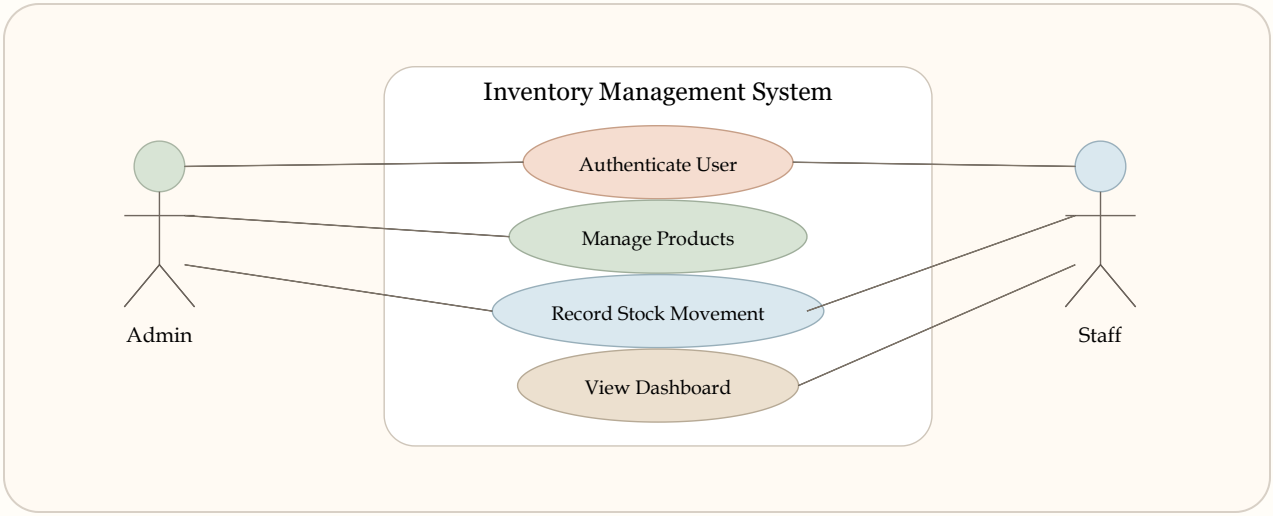
5. External Interface Requirements

- User Interface: responsive React screens for login, dashboard, and inventory operations.
- Software Interface: REST API endpoints built in Express.
- Database Interface: MongoDB collections for users, categories, products, and stock movements.
- Communication Interface: HTTP/JSON between client and server.

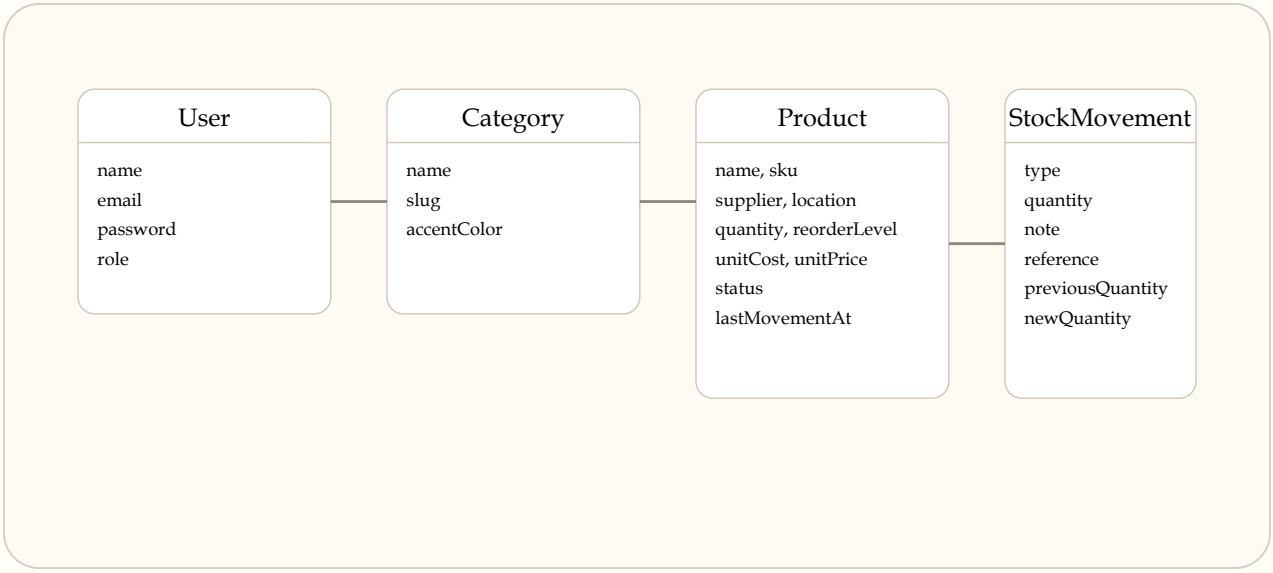
EXPERIMENT-01

Diagrams

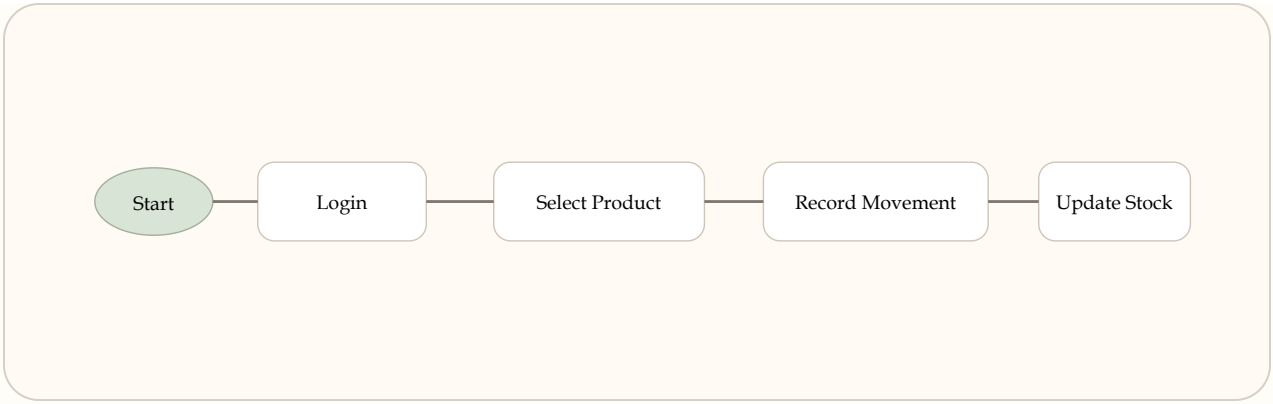
Use Case Diagram



Class Diagram



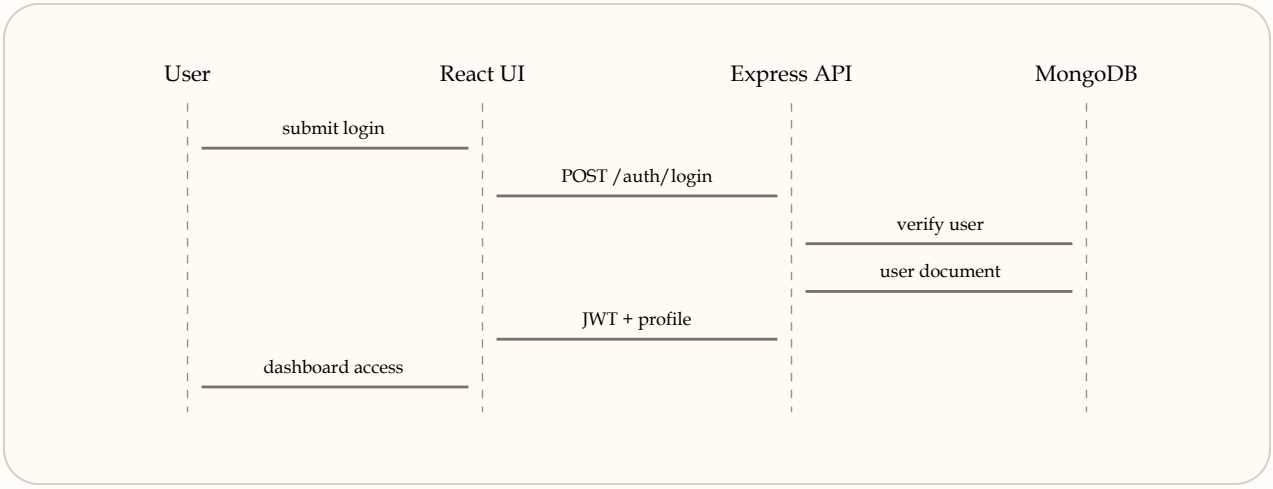
Activity Diagram


Start
Login
Select Product
Record Movement
Update Stock

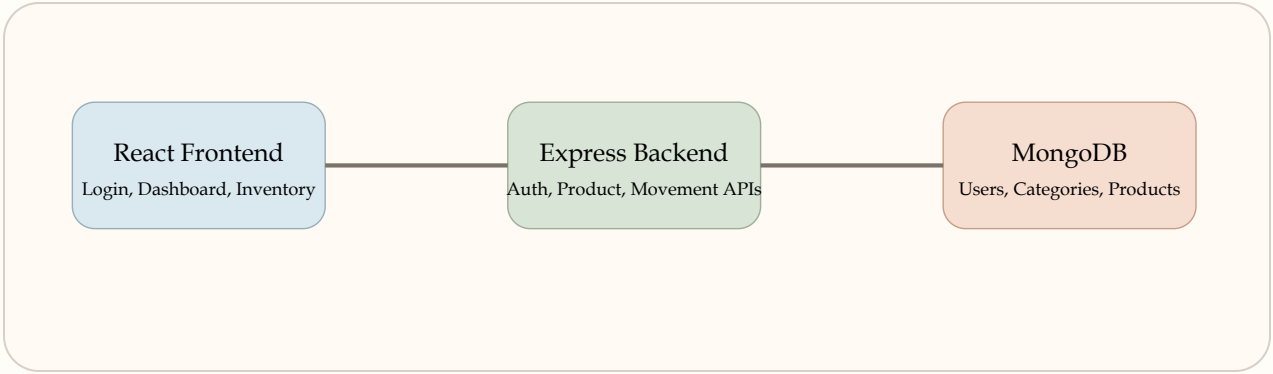
EXPERIMENT-01

Additional Architecture Diagrams

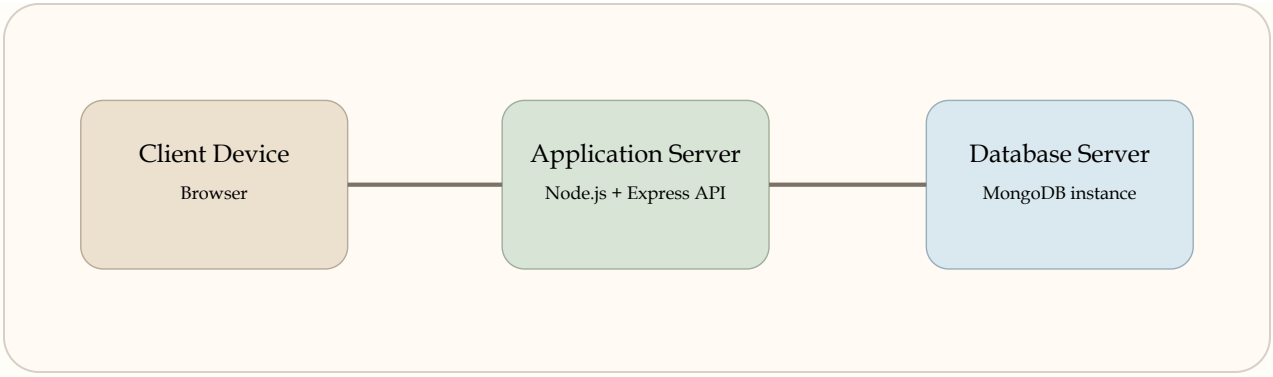
Sequence Diagram



Component Diagram



Deployment Diagram


Client Device
Browser
Application Server
Node.js + Express API
Database Server
MongoDB instance

EXPERIMENT-02

Work Breakdown Structure

The WBS decomposes the project into manageable tasks so planning, ownership, and progress tracking remain clear.

WBS ID	Task	Description
1.0	Requirement Analysis	Define problem statement, objectives, and user needs.
2.0	System Design	Prepare data model, API design, and UI wireflow.
3.0	Backend Development	Create models, routes, middleware, and authentication.
4.0	Frontend Development	Build React screens, forms, dashboard, and navigation.
5.0	Database Integration	Connect MongoDB and validate CRUD and stock movement flows.
6.0	Testing	Check usability, stock calculations, and route correctness.
7.0	Deployment Preparation	Configure environment variables, scripts, and documentation.

EXPERIMENT-03

Gantt Chart

Task	W1	W2	W3	W4	W5	W6	W7	W8
Requirement gathering								
Architecture and schema design								
Backend development								
Frontend development								
Integration and fixes								

Testing and documentation						

EXPERIMENT-04

Software Metrics

Metric	Estimated Value	Interpretation
Estimated LOC	4,500	Combined frontend, backend, and documentation code assets.
Function Points	52	Based on login, product CRUD, movement logging, dashboard summaries, and reports.
Average Response Time	Below 2 seconds	Target for regular dashboard and inventory interactions.
Productivity	28 LOC per person-day	Reflects modular development pace for a small academic team.
Defect Density Target	Below 0.8 defects/KLOC	Indicates focus on correctness in stock calculations and auth flows.

Metric Formula Examples

Productivity = Total LOC / Effort in person-days

Defect Density = Number of defects / KLOC

Inventory Value = Sum of quantity x unit cost

Quality Focus

Inventory systems are highly sensitive to quantity mismatches. Therefore, the most important metrics are stock accuracy, API correctness, and dashboard consistency after every movement event.

EXPERIMENT-05

Project Cost Estimation

Cost Head	Estimated Cost (INR)	Remarks
Requirement and design	18,000	Analysis, SRS, diagrams, and architecture planning.
Backend development	42,000	Authentication, API routes, models, business logic.
Frontend development	38,000	Responsive UI, forms, tables, and dashboard layout.
Testing and QA	14,000	Flow checks, data validation, and UI verification.
Documentation	10,000	SPM report, project notes, and deployment guidance.
Total Estimated Cost	**122,000**	**Moderate-cost academic or prototype implementation.**

EXPERIMENT-06

Resource Allocation

Resource Type	Allocated Items	Purpose
Human Resources	Project manager, MERN developer, UI designer, tester	Execution, review, quality assurance, and coordination.
Hardware Resources	Laptop/desktop, internet connection	Development, local testing, and report preparation.
Software Resources	Node.js, MongoDB, VS Code, browser, Git	Programming, storage, testing, and collaboration.
Time Resources	8-week academic schedule	Requirement analysis through final documentation.

EXPERIMENT-07

Team Structure and Roles

Role	Responsibilities	Main Deliverables
Project Manager	Planning, milestone review, coordination	Timeline, WBS, overall delivery control
Backend Developer	API, models, auth, stock logic	Express server and MongoDB integration
Frontend Developer	UI design, forms, dashboard, responsiveness	React interface and soft-tone user experience
Tester	Flow testing, validation, regression checks	Bug reports and quality review notes

Project Outcome

A modern inventory platform that improves visibility and reduces manual stock errors.

Business Benefit

Faster stock updates, better replenishment awareness, and cleaner warehouse operations.

Technical Benefit

Separation of concerns between frontend, backend, and persistence layers.

Future Scope

Barcode support, supplier purchase orders, analytics exports, and deployment automation.

Conclusion

The Inventory Management System demonstrates how a MERN-based application can be planned, implemented, and documented using standard software project management practices. The project replaces fragile manual workflows with a structured, secure, and user-friendly platform for product management and stock visibility.

Through SRS preparation, UML diagrams, WBS, Gantt planning, metrics, cost estimation, and team structuring, the project provides both a practical software solution and a complete academic documentation package.